Session 9: How a Neural Network Learns, and Why Training Fails

KB8029 — Bioinformatics & Deep Learning

Arne Elofsson

SciLifeLab

SciLifeLab

Log in to the lecture quiz

Log in to the quiz

Scan the code, or go to quiz.bioinfo.se/live

Log in with your university account

Answering counts as being at the lecture: up to 3 bonus points on the exam

Right or wrong is never graded, only whether you answered. Results are shown without names.

quiz.bioinfo.se/live

SciLifeLab

Warm-up: what do you know already?

3 quick questions

On the topics of today, one at a time, about 30 seconds each

Answer on your own, and guess if you do not know

It shows me where to start; right or wrong is never graded

SciLifeLab

The chain rule: if y = f(g(x)), then dy/dx is ...

A $f'(x) \cdot g'(x)$

B $f'(g(x)) \cdot g'(x)$

C $f'(x) + g'(x)$

D $f(g'(x))$

Answer now

Scan the code, or go to

quiz.bioinfo.se/live

SciLifeLab

True label 1, prediction p = 0.01. The cross-entropy loss is −ln p. About how large?

A 0.01

B 0.99

C 4.6

D 100

Answer now
Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

Training accuracy 100%, validation accuracy 60%. What is this?

A Underfitting

B Overfitting

C A learning rate that is too small

D A vanishing gradient

Answer now
Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

What the room knows already

Where we start

Which topics most of you know, and which we take slowly today.

Show the results ▸ quiz.bioinfo.se/live/report

Your questions before today

What you asked

The questions you sent in on Canvas before the lecture

The most common ones now; the others in the block where they belong

SciLifeLab

In up to three words: what do you want to learn today?

Training a network

loss, backprop, mini-batches

Why training fails

initialisation, vanishing gradients, epistasis

Overfitting

L1, L2, dropout, is it real?

Answer now

Scan the code, or go to
quiz.bioinfo.se/live



SciLifeLab

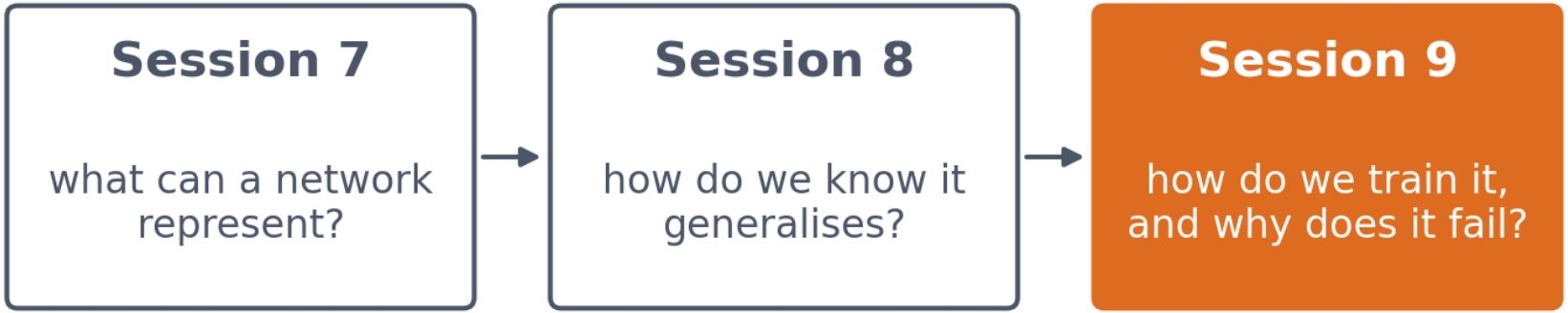
What you want to learn today

What the room wants to learn

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

Session 8's network had random weights. How do we make it useful?



Sessions 7, 8 and 9 form one progression: what a network can represent, how we know it generalises, and how we actually train it.

See course.bioinfo.se/session09.html.

SciLifeLab

Three blocks: about 30 min talk and 15 min discussion



Block 1: the training loop, backpropagation and the loss. Block 2: why training fails, and when a hidden layer pays off. Block 3: overfitting, regularisation, and whether the improvement is real.

Times are approximate: a 10 min start (warm-up quiz and your questions), two 15 min breaks and a 5 min close. Each block ends with a quiz question: guess, run the notebook and work out why it behaves as it does, discuss, answer again.

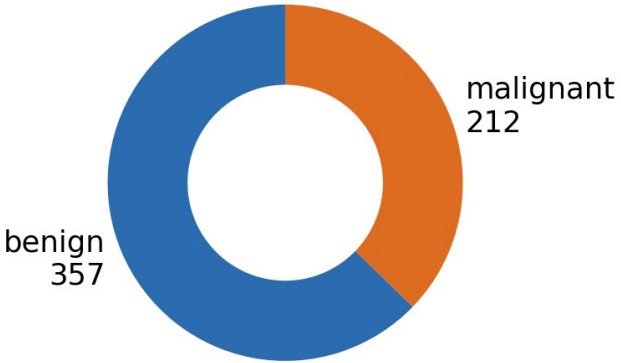
SciLifeLab

BLOCK 1 · ABOUT 30 MIN

From an untrained network to a trained one

Breast cancer: 569 biopsies

569 biopsies, 30 features each



split once; standardised with training statistics

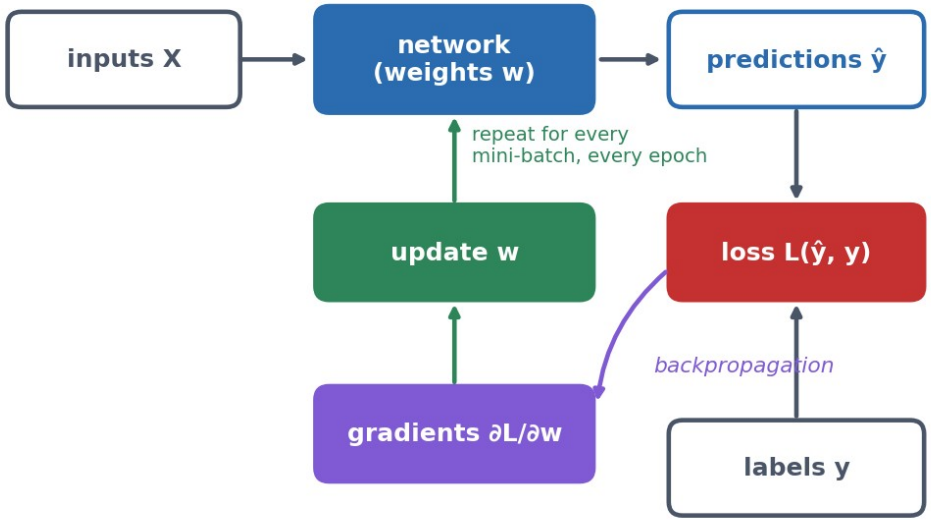


- **30 features per biopsy**
- Size and shape of the nuclei
- Network 30 → 16 → 1
- 513 weights

Breast Cancer Wisconsin (Diagnostic), UCI, via scikit-learn; notebook Section 1.

SciLifeLab

Predict, measure the loss, compute gradients, update, repeat



The training loop in PyTorch

`optimizer.zero_grad()`
1. clear the old gradients

`logits = model(x)`
2. forward pass: predictions

`loss = criterion(logits, y)`
3. how wrong are they?

`loss.backward()`
4. backprop: all gradients

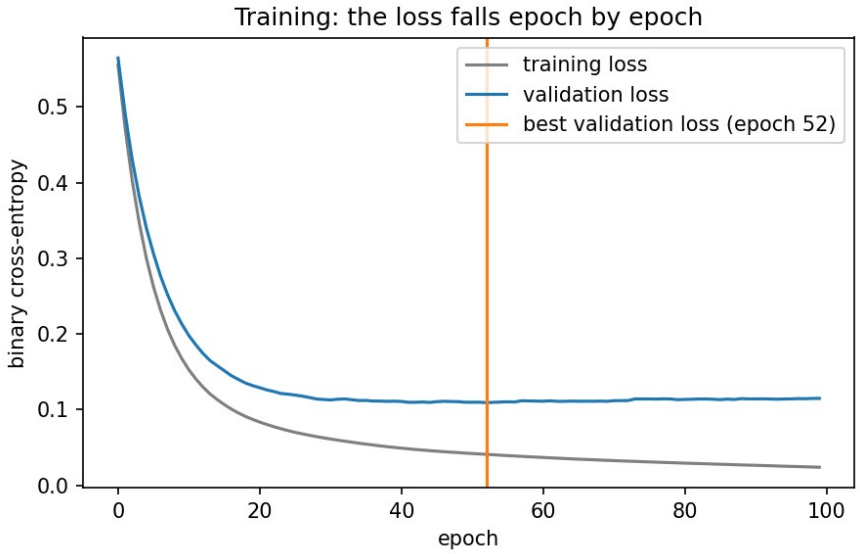
`optimizer.step()`
5. update every weight

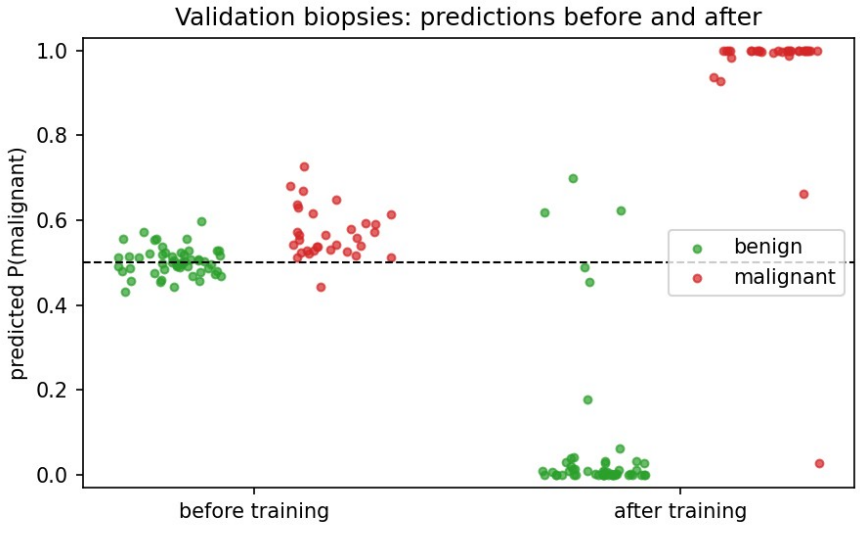
The same five lines run for every mini-batch in every epoch. You write this loop yourself in the lab.

See course.bioinfo.se/session09.html, 'From an untrained network to a trained network'.

SciLifeLab

Training turns guesses into predictions

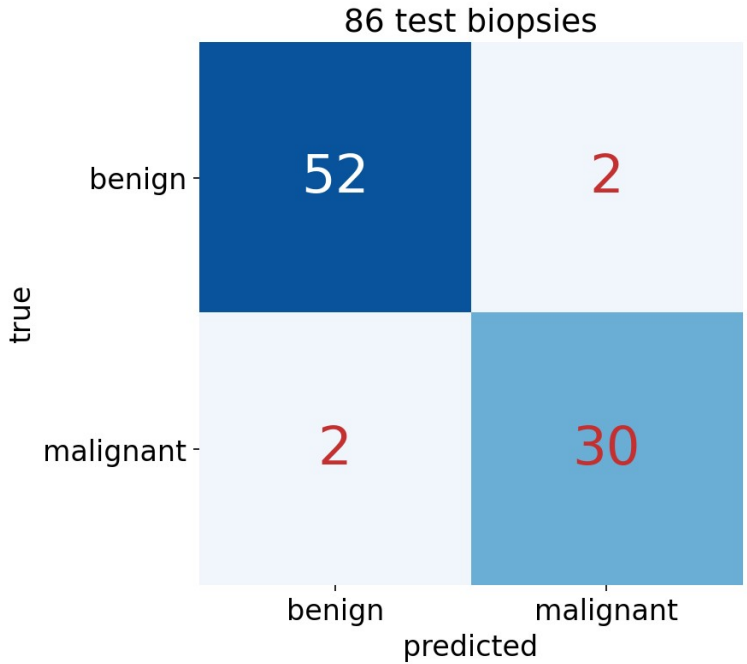




Left: the loss per epoch; the validation loss is lowest at epoch 52 (early stopping). Right: before training every prediction sits near 0.5; after, benign go to 0 and malignant to 1.

SciLifeLab

On the test set: 4 mistakes in 86

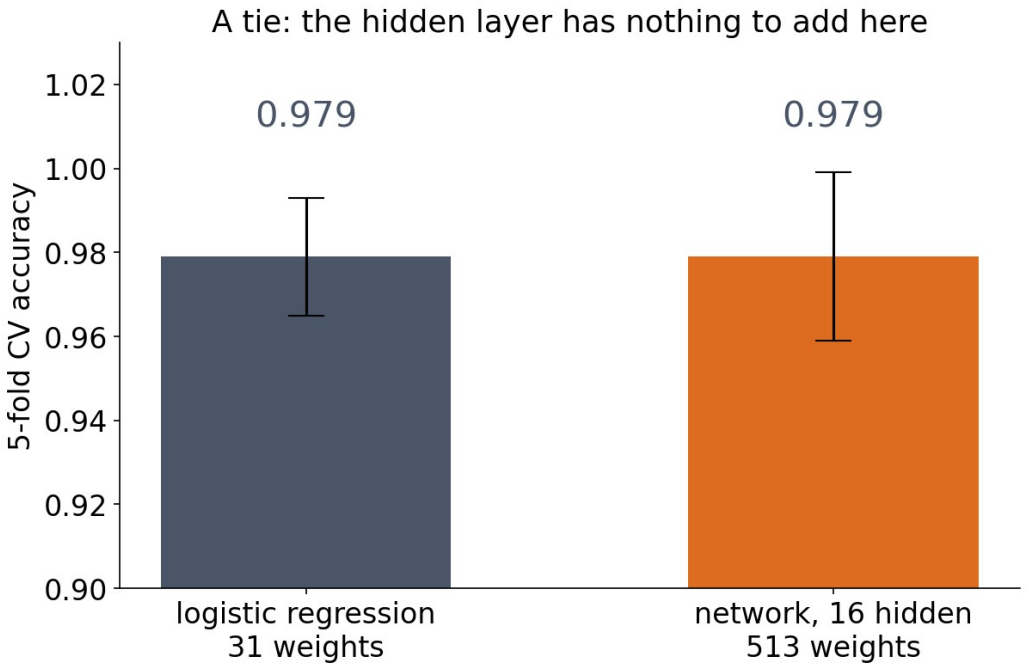


- **Accuracy 0.953**
- **MCC 0.900**
- **AUROC 0.984**
- 2 false alarms, 2 missed

Notebook, Section 1.

SciLifeLab

Was the hidden layer needed? No

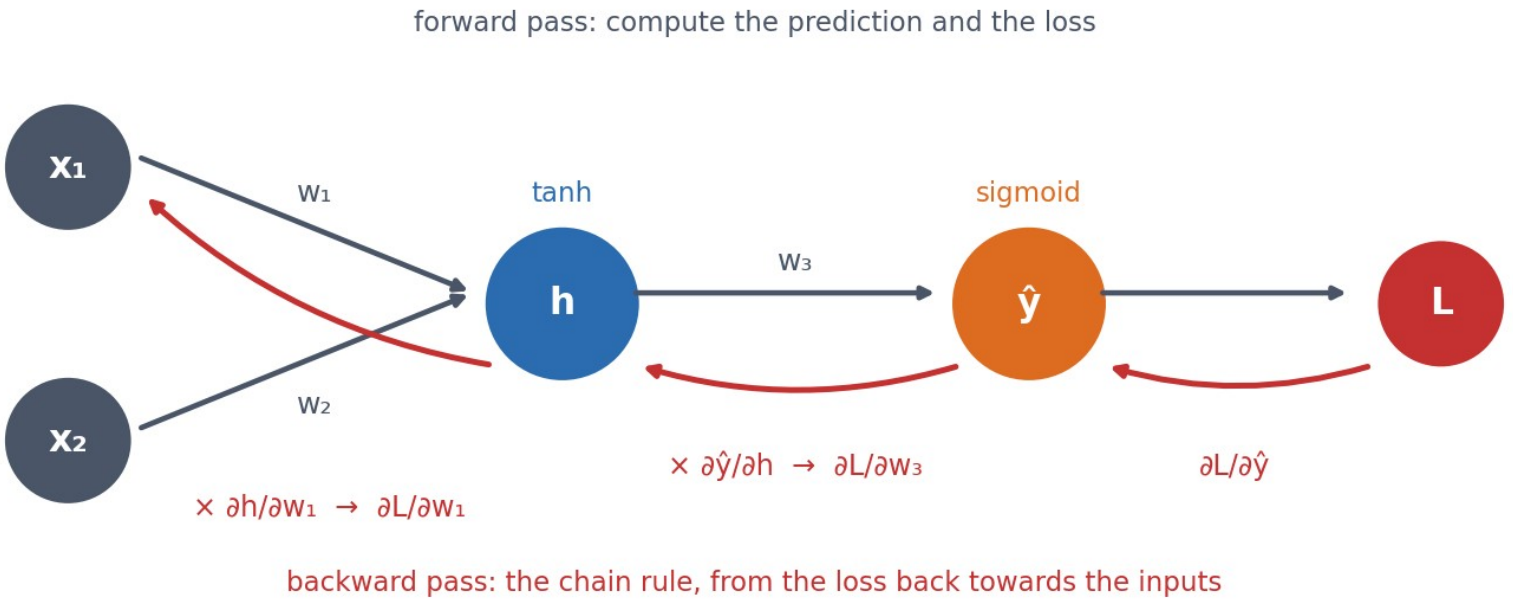


- Same accuracy with 31 weights
- Nearly additive data
- **Training works; capacity adds nothing**

Notebook, Section 1: 5-fold cross-validation.

SciLifeLab

How does the network know which weights to change?



The chain rule, from the loss backwards: $\partial L/\partial \hat{y}$ is computed once and reused for w_3, then multiplied by $\partial \hat{y}/\partial h$ and reused for w_1 and w_2.

See course.bioinfo.se/session09.html, 'How backpropagation finds every gradient'.

SciLifeLab

By hand and by autograd: the same five gradients

weight	chain rule by hand	loss.backward()	match?
w_xg0	-0.06640098	-0.06640099	yes
w_xg1	+0.05976089	+0.05976089	yes
w_xg2	-0.00664010	-0.00664010	yes
w_gf0	-0.14127646	-0.14127646	yes
w_gf1	+0.03460124	+0.03460124	yes

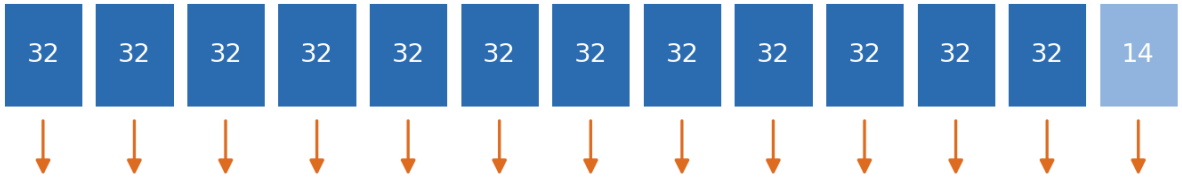
The notebook differentiates the tiny network by hand with the chain rule, then calls loss.backward(). They agree to eight decimal places; from now on autograd does the work.

Notebook, Section 2.

SciLifeLab

Mini-batches: many cheap steps per epoch

one epoch = 398 training biopsies, shuffled, in mini-batches of 32



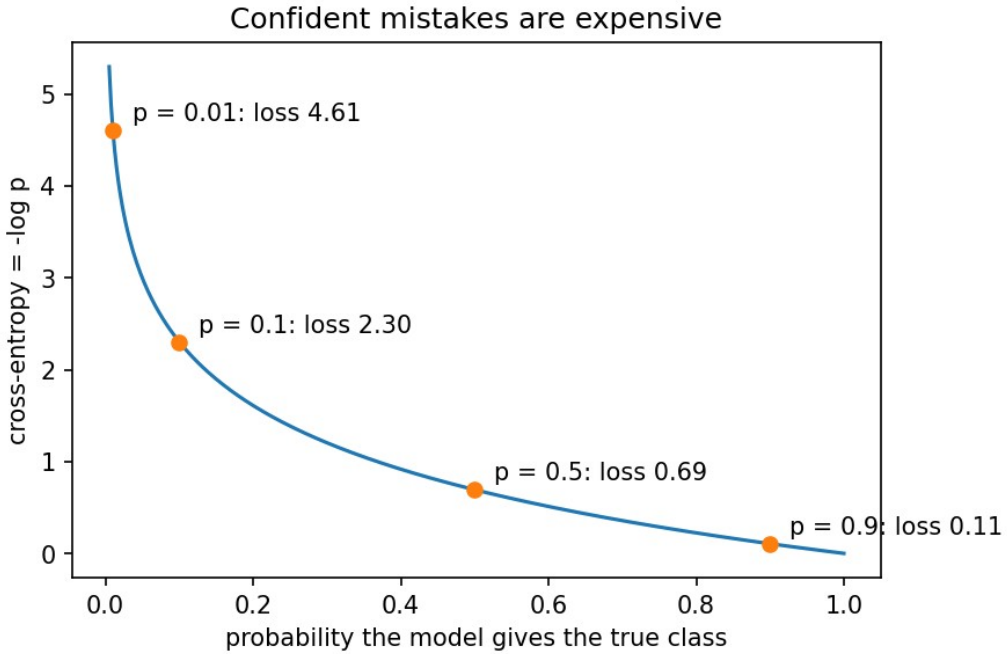
13 weight updates per epoch: many cheap, slightly noisy steps

- **Stochastic gradient descent**
- Shuffle every epoch
- Validate after every epoch

See course.bioinfo.se/session09.html; DataLoader with batch size 32.

SciLifeLab

Cross-entropy: confident mistakes are expensive

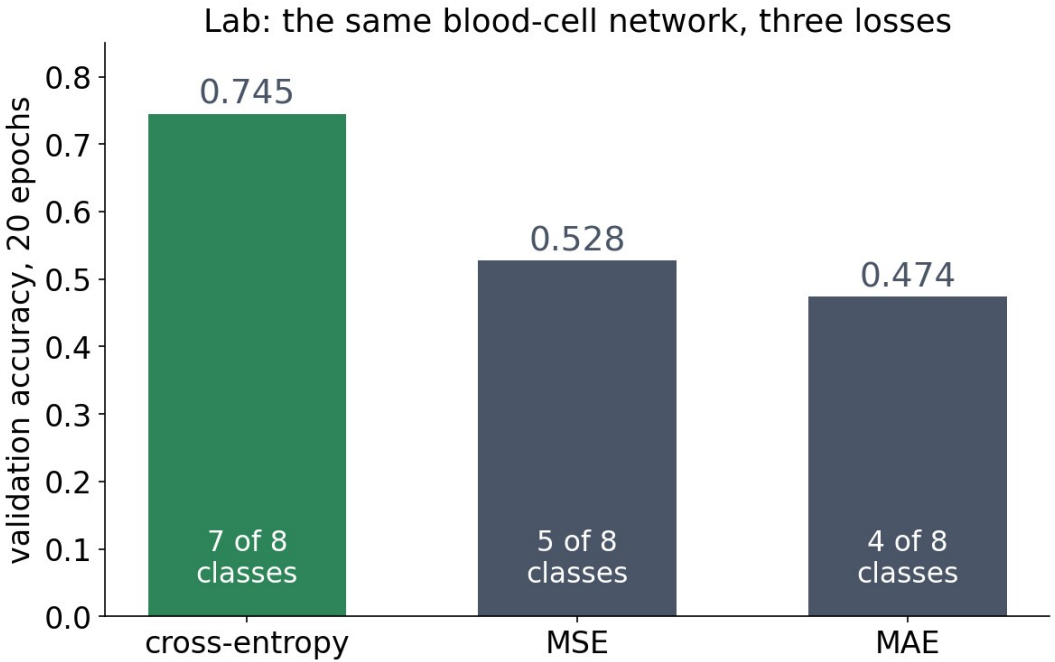


- **CE = −log p(true class)**
- p = 0.9 costs 0.105
- p = 0.01 costs 4.6
- Regression: mean squared error

See course.bioinfo.se/session09.html, 'Which loss to minimise'.

SciLifeLab

In the lab: cross-entropy trains best



- Same network, same data
- **MSE and MAE give up on classes**
- Cross-entropy pushes classes apart

Session 9 lab, executed solution (BloodMNIST, 8 classes).

SciLifeLab

What is w1.grad?

Two inputs, one ReLU hidden unit, one sigmoid output. The hidden unit's input is z1 = −0.52.

```
x1, x2 = 0.5, -0.3
w1, w2, b1 = 0.2, 0.4, -0.5     # hidden unit
w3, b2 = 0.9, -0.1              # output unit
# (the weights: tensors, requires_grad=True)
z1 = w1*x1 + w2*x2 + b1         # -0.52
a1 = torch.relu(z1)
y_hat = torch.sigmoid(w3*a1 + b2)
loss = -torch.log(y_hat)        # 0.744
loss.backward()
print(w1.grad)
```

A About −0.24: the error flows back to w1

B Exactly 0: ReLU passes nothing back for z < 0

C Small but not zero, as with tanh

D −0.52, the value of z1

Guess on your own

Scan the code, or go to

quiz.bioinfo.se/live

What is w1.grad?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

What is w1.grad?

Fix it, run it, discuss

Open the notebook: scan the code

Fix the line marked FIX, then run the cells

Did it come out as you guessed? Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-1.ipynb

SciLifeLab

What is w1.grad?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

What is w1.grad?

Results, and who explains

Both rounds side by side.

Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

To check it with code afterwards: discussion notebook 1 https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-1.ipynb

SciLifeLab

The answer: exactly 0; ReLU passes no gradient for z < 0

gradient	by hand, ReLU′ = 1	by hand, ReLU′ = 0 for z < 0	autograd
w_1	−0.236	0	0
w_2	+0.142	0	0
b_1	−0.472	0	0
w_3	0	0	0
b_2	−0.525	−0.525	−0.525

$z_1 = -0.52 < 0$, so ReLU passes no gradient back: w_1, w_2 and b_1 do not move, however large the loss (0.744). Only b_2 learns on this example.

The chain rule multiplies by ReLU's local derivative, which is 0 for $z_1 < 0$. So w_1, w_2 and b_1 get gradient exactly 0, and w_3 too ($a_1 = 0$); only b_2 learns. The notebook's hand calculation assumed a derivative of 1 and got −0.236: comparing with autograd catches it.

SciLifeLab

BLOCK 1 DONE

Break

SciLifeLab

BLOCK 2 · ABOUT 30 MIN

Why training fails, and when a hidden layer pays off

SciLifeLab

GB1: a protein fitness landscape

GB1: four positions, all 20^4 combinations



160,000 variants made

149,361 with a measured fitness

(IgG binding, wild type VDGV = 1)



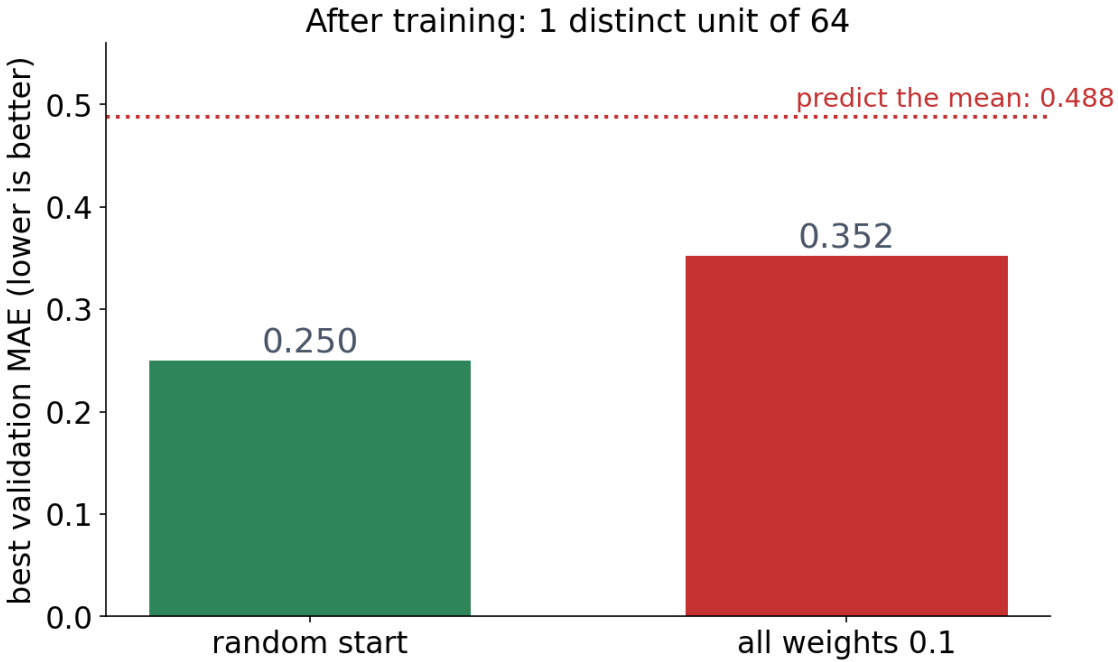
input: 4 × 20 = 80
one-hot numbers

- **Predict $\log_{10}$ fitness**
- 20,000 variants for training
- Mean predictor: MAE 0.488
- Working network: 0.250

Wu et al. (2016) eLife 5:e16965, via FLIP; notebook Section 4.

SciLifeLab

Every weight starts the same

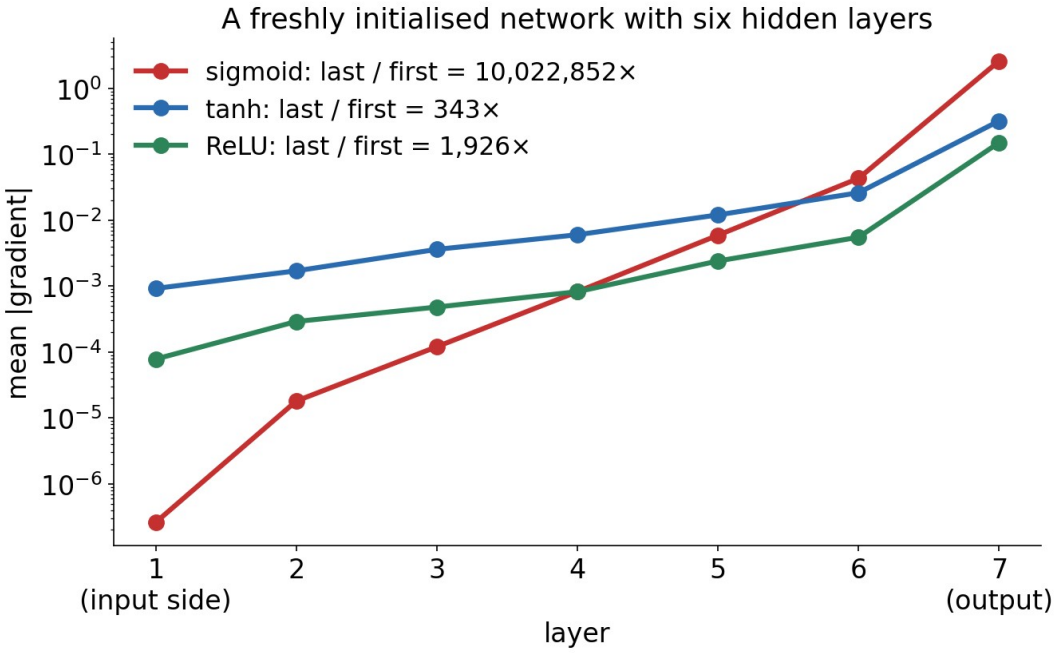


- **Same weights → same gradient**
- 64 units behave like one
- **Fix: random initialisation**

Notebook, Section 4.

SciLifeLab

Deep sigmoid networks: the gradient vanishes

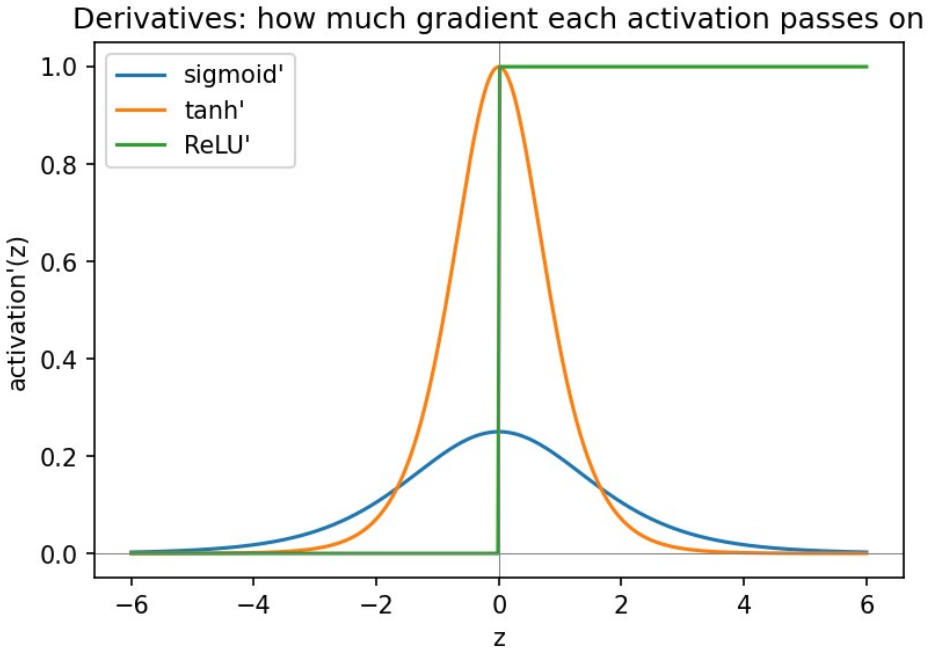


Backpropagation multiplies one activation derivative per layer. With sigmoid, the first layer of a freshly initialised six-layer network gets ten million times less gradient than the last.

Notebook, Section 4.

SciLifeLab

How much gradient does each activation pass on?

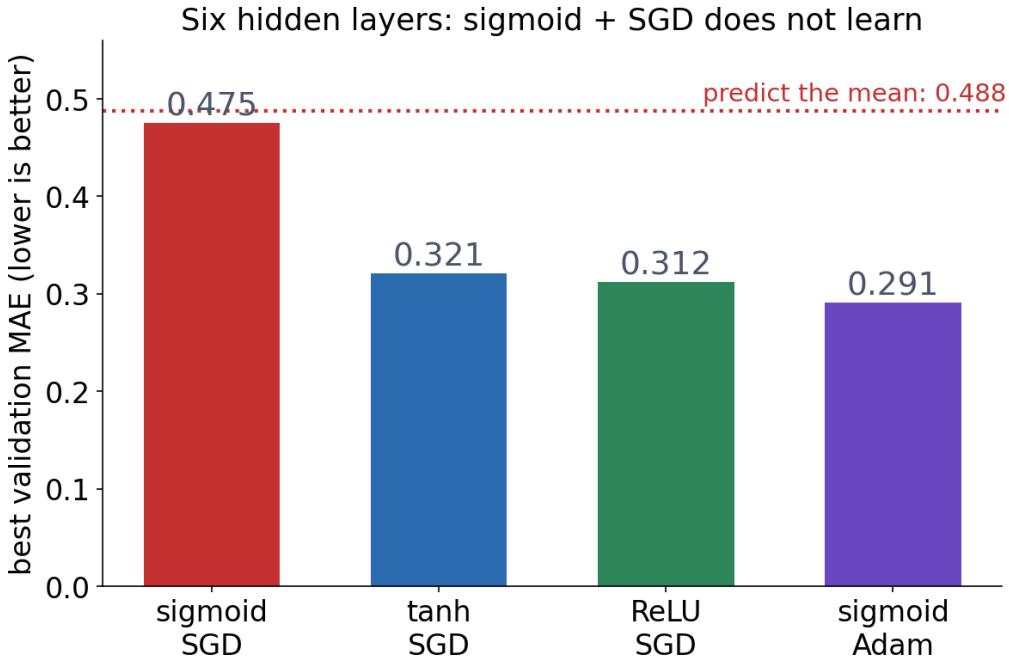


- **Sigmoid: at most 0.25**
- tanh: 1 at 0, vanishes in the tails
- **ReLU: exactly 0 or 1**

Notebook, Section 4.

SciLifeLab

Six sigmoid layers do not learn at all

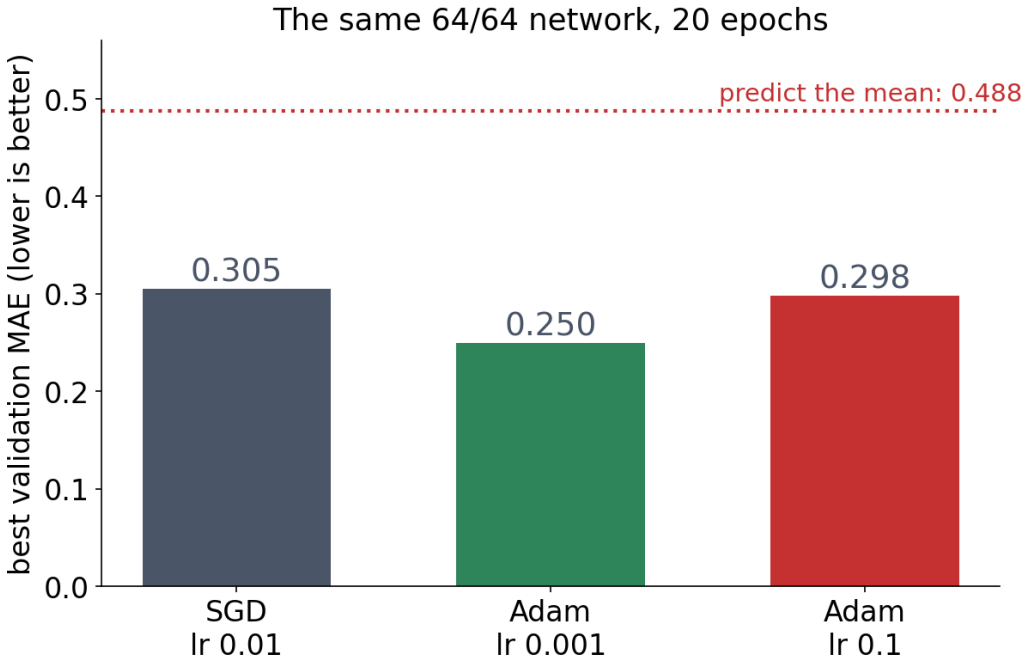


- **Fix: change what the gradient passes through**
- ReLU, scaled initialisation
- Normalisation, residual connections
- Adam: a workaround, not a cure

Notebook, Section 4; Ioffe & Szegedy (2015) for batch normalisation.

SciLifeLab

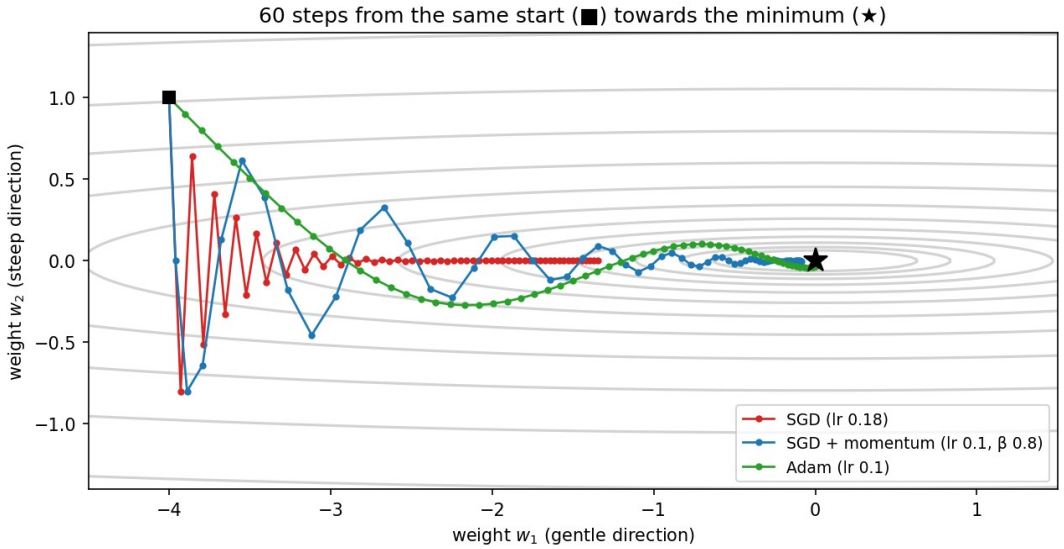
The optimiser and the learning rate

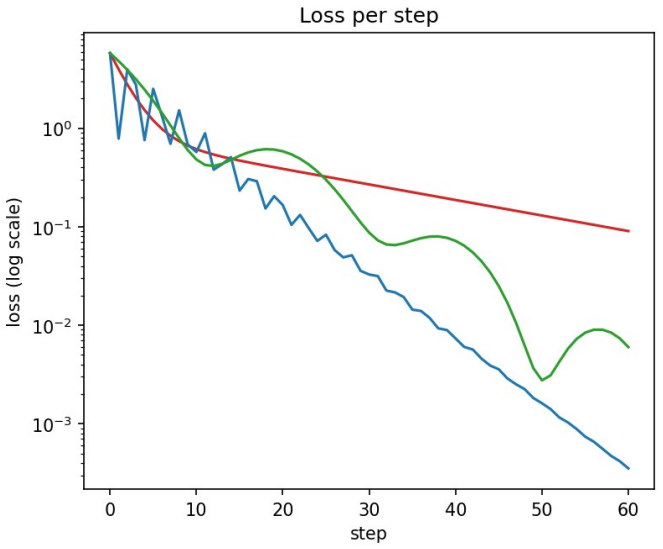


- Too large: jumps around
- Plain SGD: one step size for all
- **Start with Adam, lr ≈ 10^{-3}**
- **Always watch the validation curve**

Notebook, Section 4.

Why can an optimiser beat plain SGD?



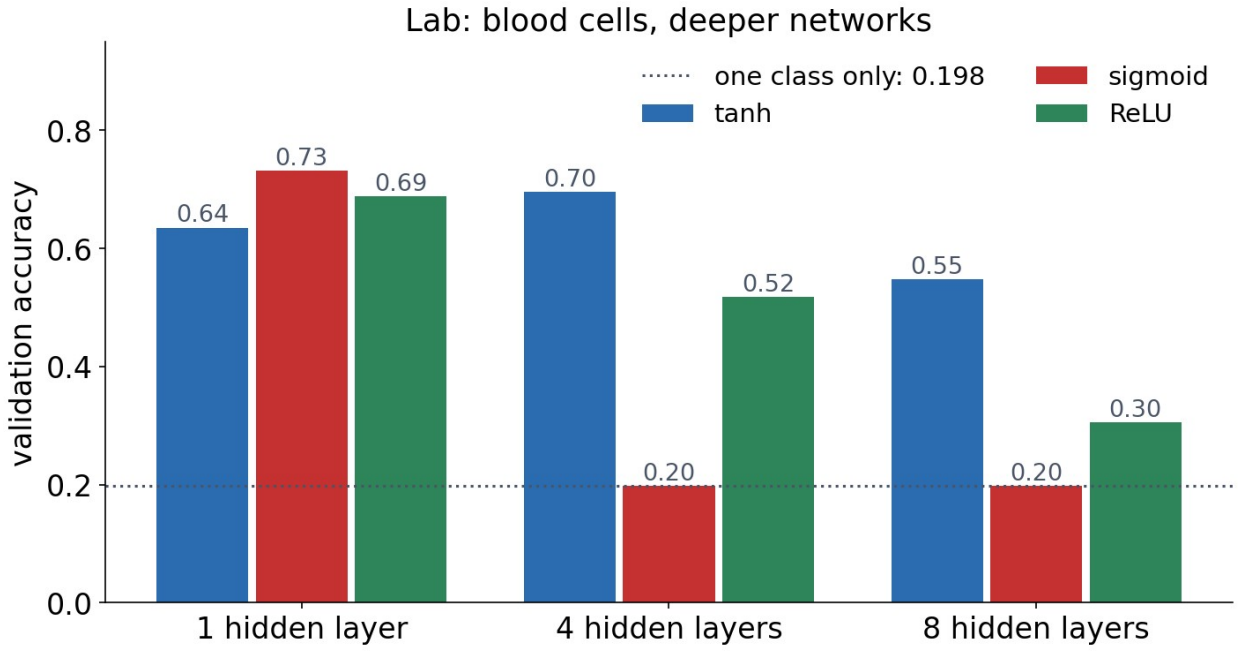


A valley 100× steeper across than along. SGD zigzags, then crawls (loss 0.09 after 60 steps). Momentum averages recent directions (0.0004). Adam adds a step size per weight (0.006).

Notebook, Section 4; Kingma & Ba (2015).

SciLifeLab

The lab repeats the failures on blood cells



With 4 or 8 sigmoid layers the network predicts a single class (0.198). The sigmoid network's first-layer gradient is 2 million times smaller with 8 hidden layers than with 1.

SciLifeLab

Three ways training fails

Equal start

units stay identical

- Same gradient for every unit
- Fix: random initialisation

Deep sigmoid

gradient vanishes

- ≤ 0.25 per layer
- Fix: ReLU, init, normalisation, residuals

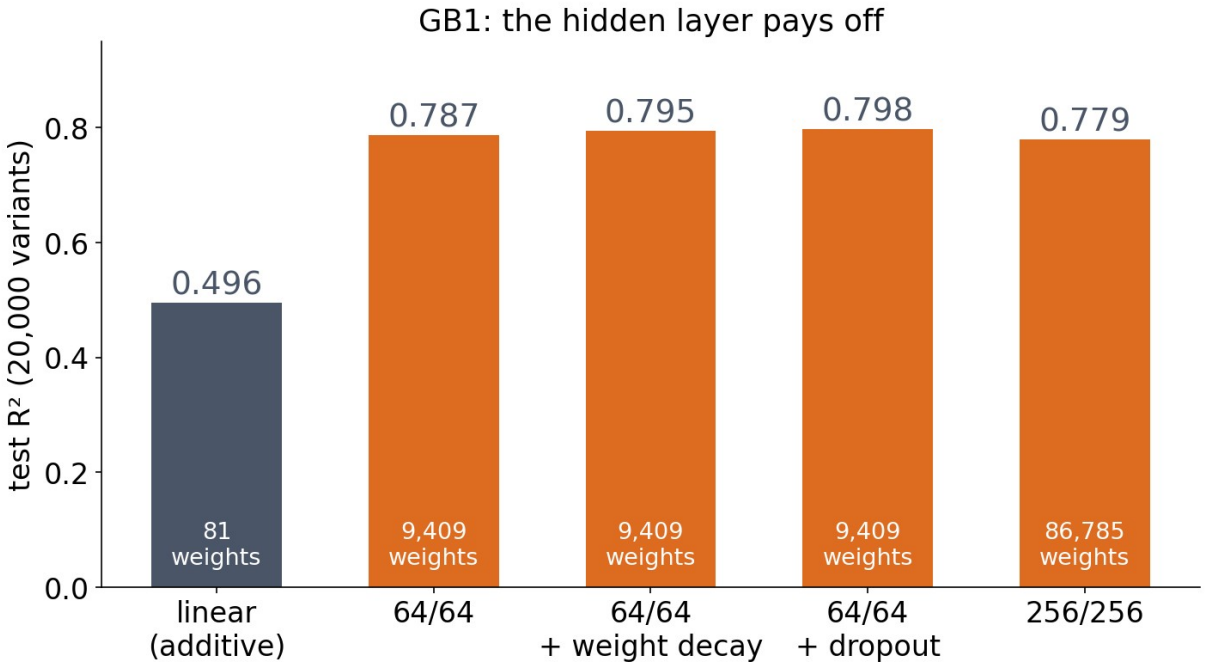
Learning rate

overshoot or crawl

- One step size for all weights
- Fix: Adam, lr ≈ 10^{-3}, watch validation

SciLifeLab

GB1: the network explains 79%, the sum 50%

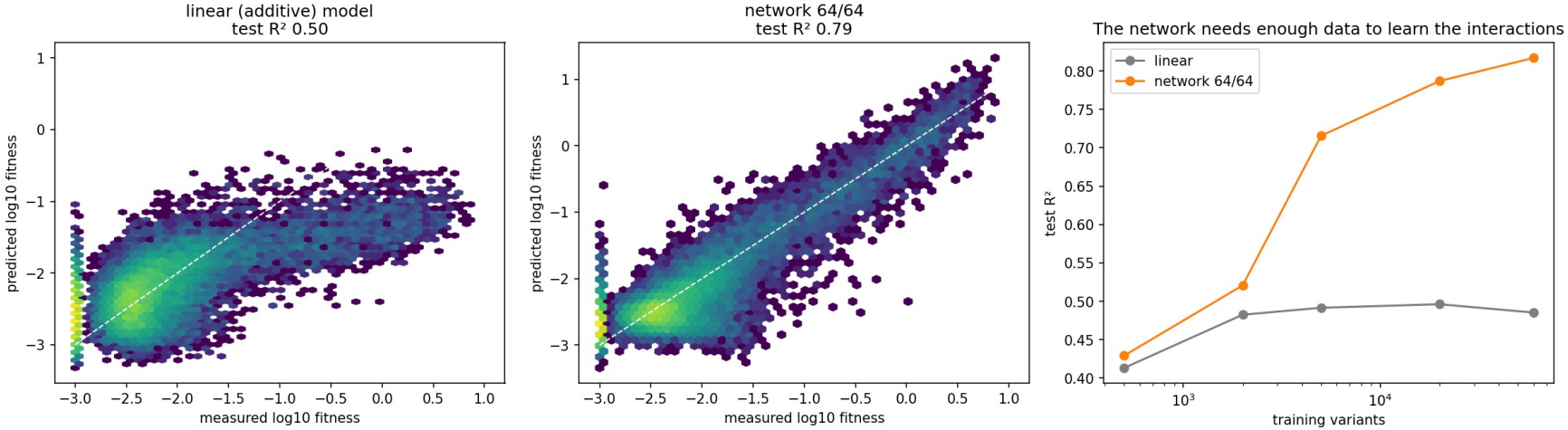


- **Linear on one-hot = a PSSM**
- Cannot represent epistasis
- Ten times larger: no better

Notebook, Section 5; see the table on course.bioinfo.se/session09.html.

SciLifeLab

The additive model compresses; the network follows the diagonal

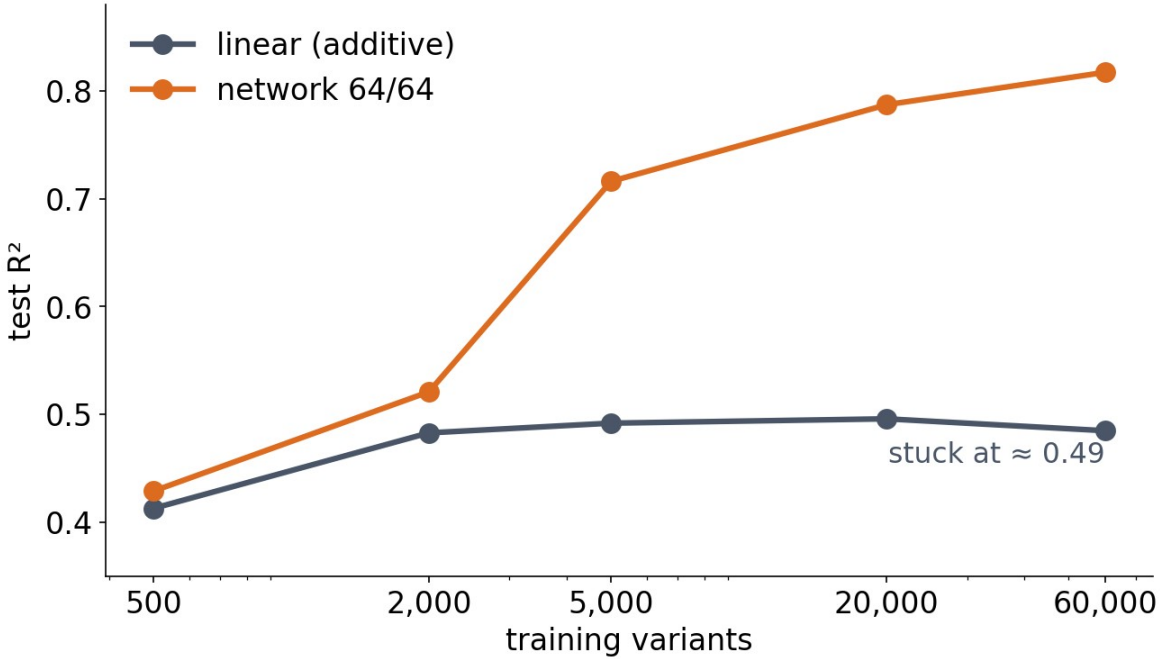


Predicted against measured $\log_{10}$ fitness for 20,000 test variants: additive model (left, R^2 0.50) and 64/64 network (middle, R^2 0.79). Right: test R^2 against training-set size.

Wu et al. (2016); notebook Section 5.

SciLifeLab

The network needs data to learn the interactions



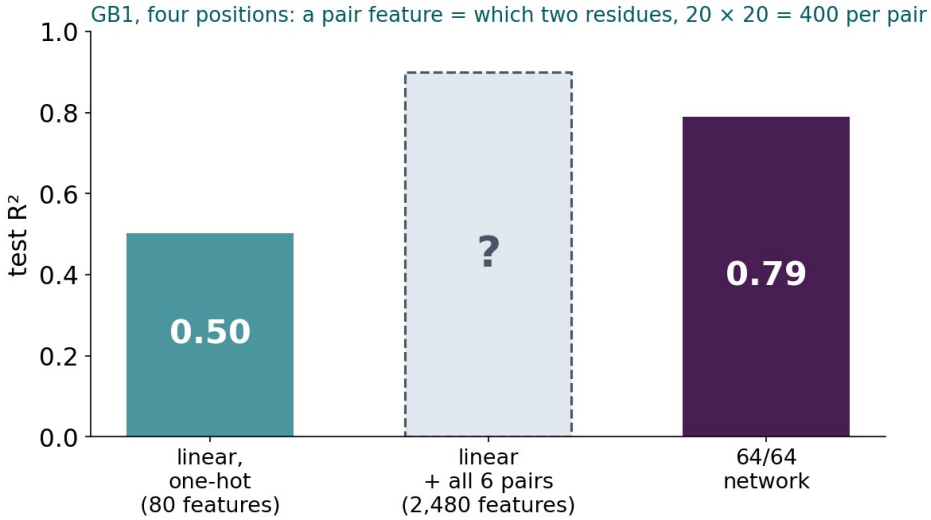
- 500 variants: a tie
- **Thousands: the network pulls away**
- No data lets a sum represent an interaction

Notebook, Section 5.

SciLifeLab

What R^2 does the linear model reach with pair features?

GB1: linear on one-hot 0.50, 64/64 network 0.79. Add 400 features for each of the six pairs of positions.



A About 0.50: still linear, nothing changes

B About 0.74: most of the way to the network

C About 0.79 or above: matches the network

D Below 0.50: 2,480 features overfit

Guess on your own

Scan the code, or go to

quiz.bioinfo.se/live

SciLifeLab

What R^2 does the linear model reach with pair features?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

What R^2 does the linear model reach with pair features?

Fix it, run it, discuss

Open the notebook: scan the code

Fix the line marked FIX, then run the cells

Did it come out as you guessed? Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-2.ipynb

SciLifeLab

What R^2 does the linear model reach with pair features?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

What R^2 does the linear model reach with pair features?

Results, and who explains

Both rounds side by side.

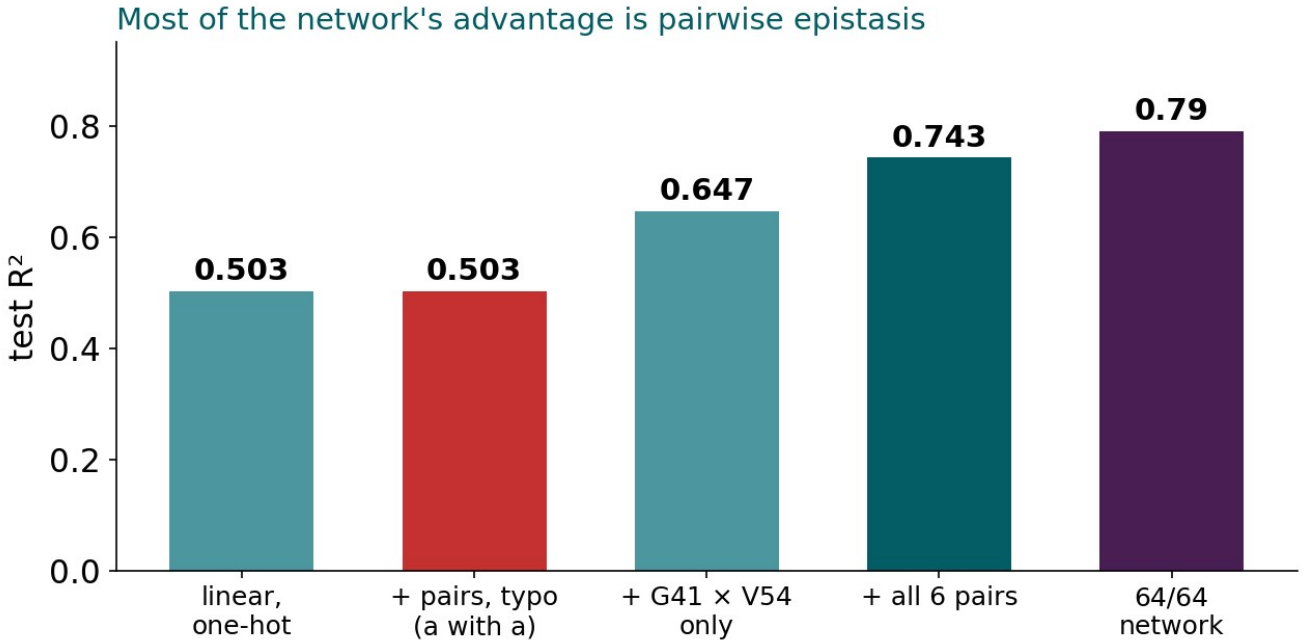
Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

To check it with code afterwards: discussion notebook 2 https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-2.ipynb

SciLifeLab

The answer: about 0.74; most of the advantage is pairwise epistasis



Pair features take the linear model from 0.50 to 0.74, most of the way to the network's 0.79; G41 × V54 alone gives 0.65. Listing interactions works for four positions, not for a whole protein: a hidden layer learns the combinations that matter instead of listing them all.

SciLifeLab

BLOCK 2 DONE

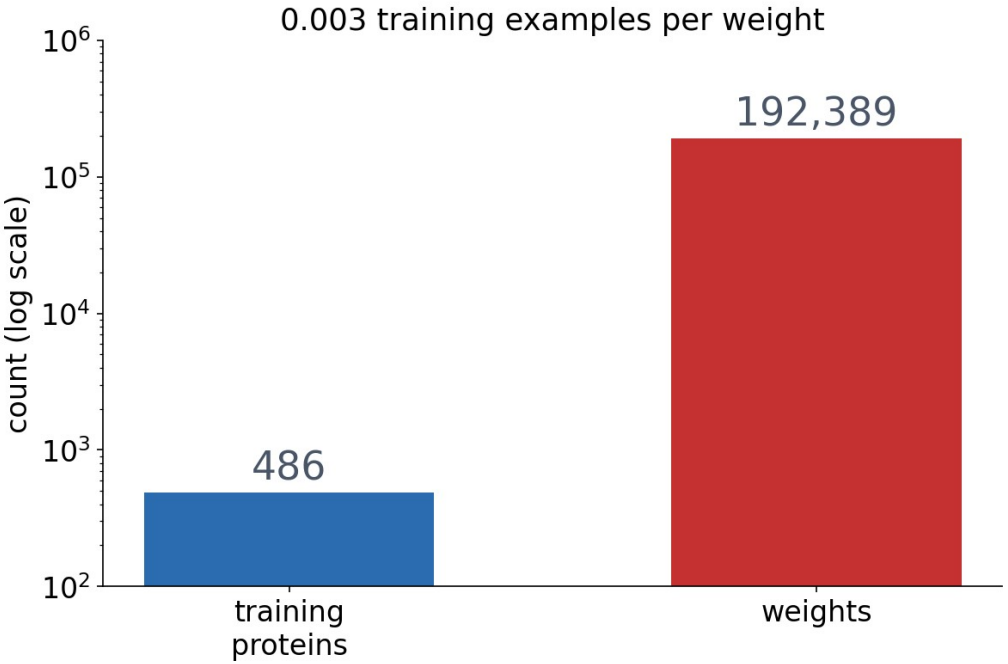
Break

SciLifeLab

BLOCK 3 · ABOUT 30 MIN

Overfitting, and is the improvement real?

Session 8's localisation network: far more weights than data

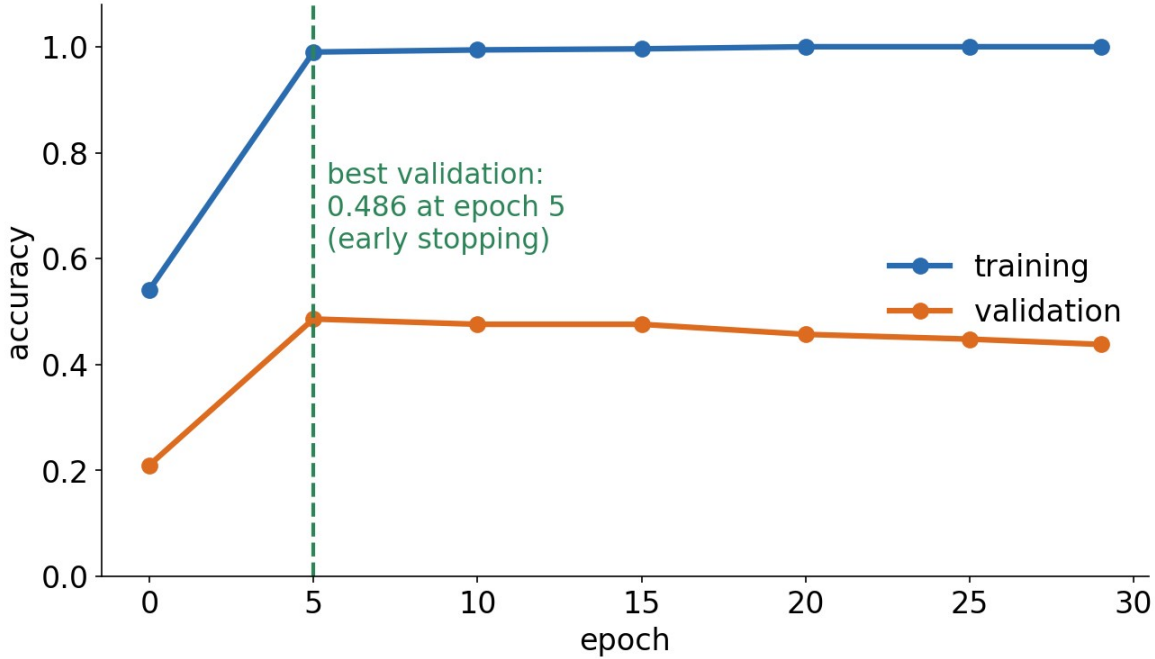


- 3,000 inputs: 150 residues, one-hot
- 64 hidden units, 5 classes
- **486 proteins to learn from**

Notebook, Section 6; homology-aware split of Session 8.

SciLifeLab

It memorises every training protein

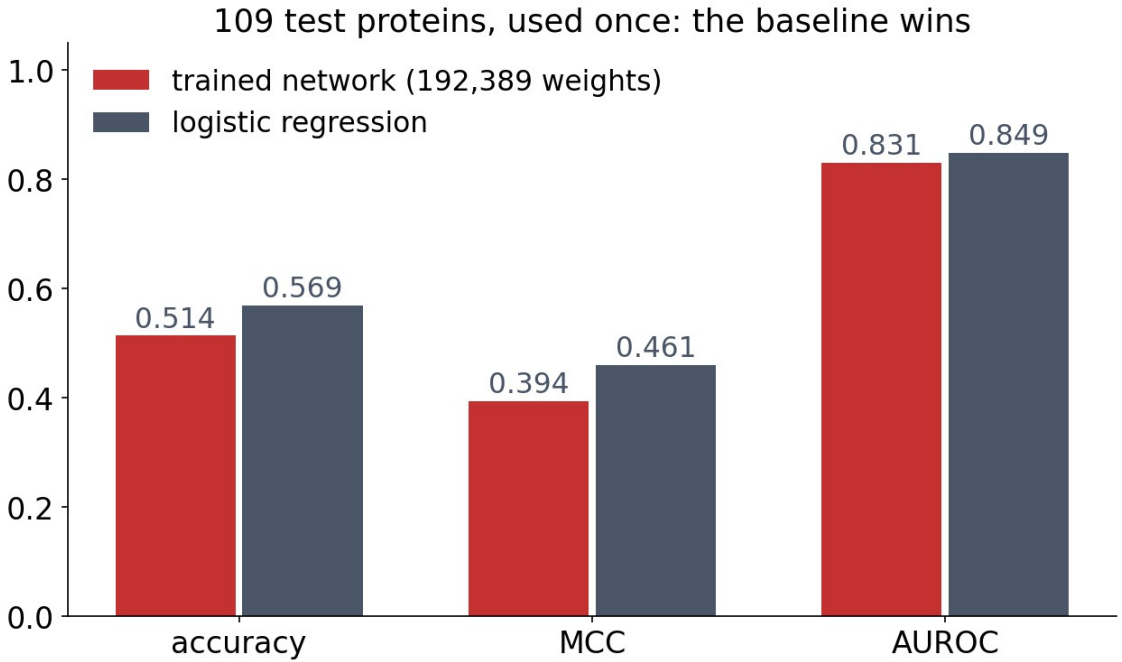


- **Training accuracy 1.000**
- Validation peaks at epoch 5
- Then falls, to 0.438
- **Overfitting in its plainest form**

Notebook, Section 6.

SciLifeLab

The simple baseline wins



- Every number far above chance
- **Logistic regression is better**
- Always compare with a baseline

Notebook, Section 6; test set used once.

SciLifeLab

Five remedies

- **More data: the most reliable fix**
- A smaller model or a better architecture
- Early stopping
- Weight decay (L2) or L1
- Dropout

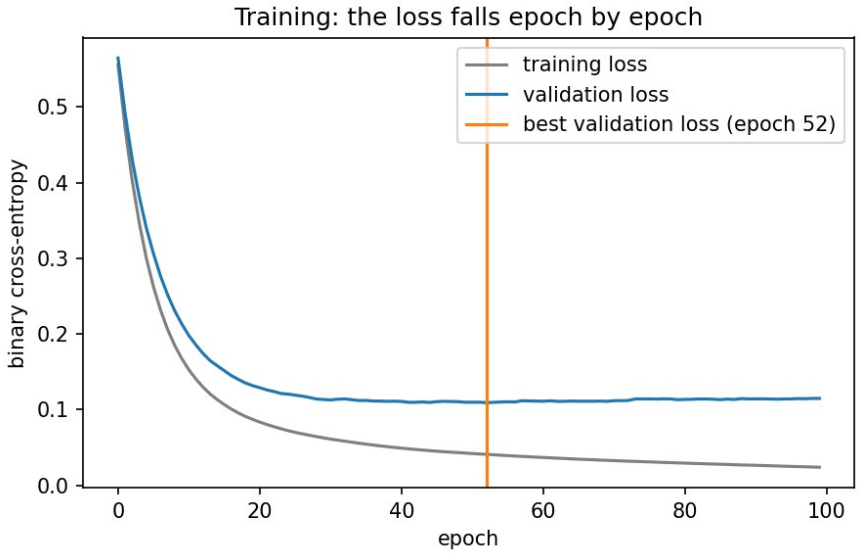
0.486
early stopping
(best validation)

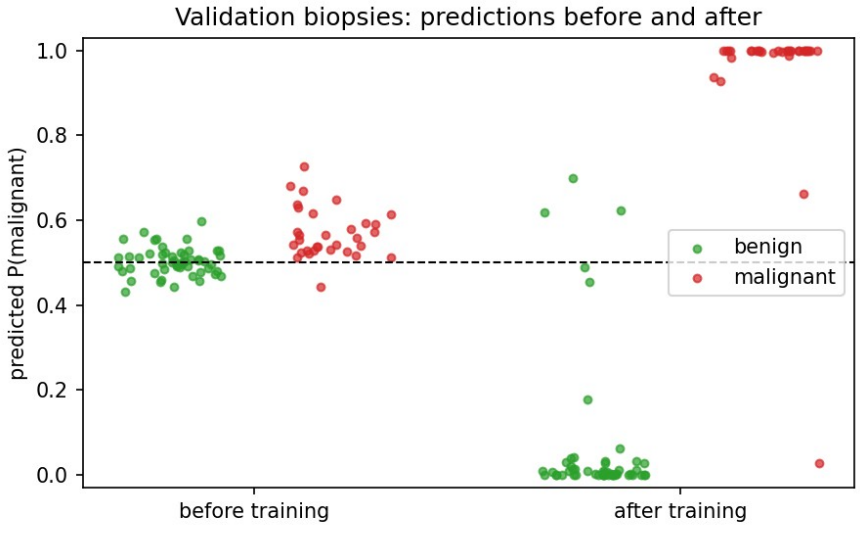
0.476
+ dropout and
weight decay

See course.bioinfo.se/session09.html, 'What can we do about overfitting?'.

SciLifeLab

Two curves, two questions

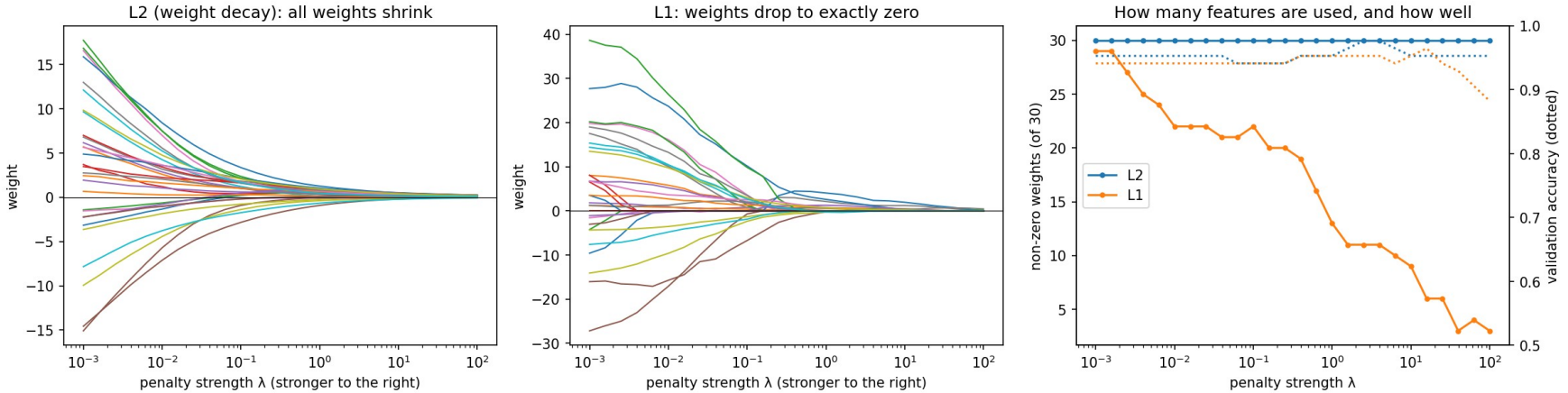




Training loss: is optimisation working? Validation loss: does more optimisation still help generalisation? Keep the epoch with the best validation loss (here 52), and judge it on the test set once.

See course.bioinfo.se/session09.html, 'Early stopping'.

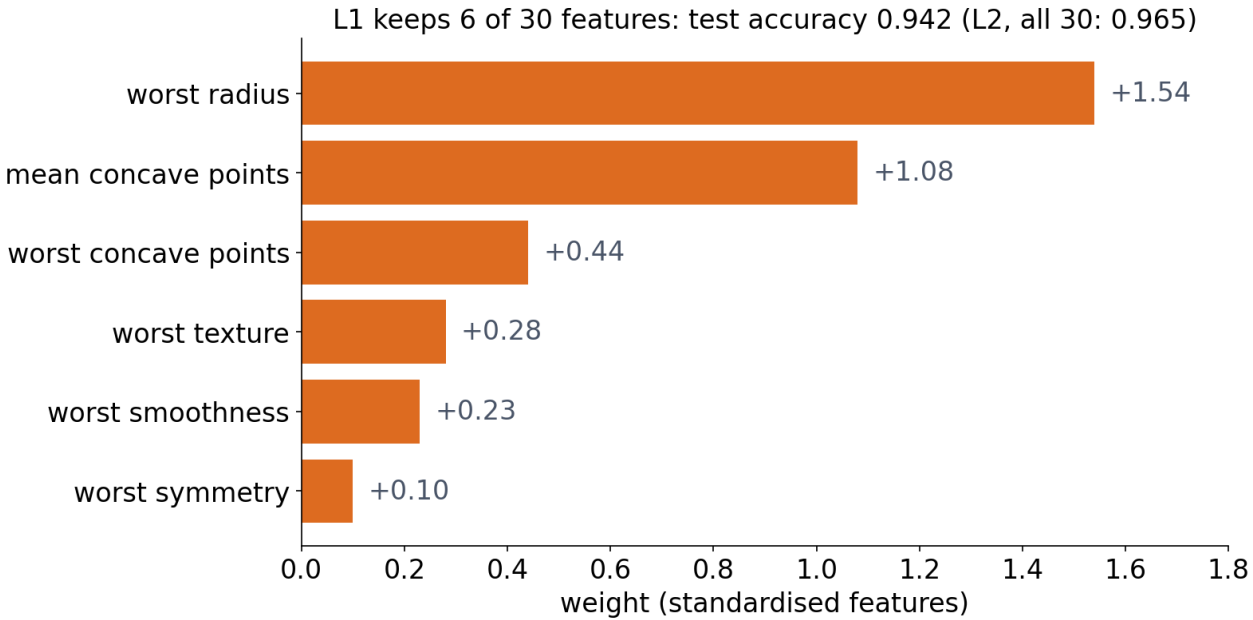
L2 shrinks every weight; L1 sets many to exactly zero



Loss + λΣw² (L2, weight decay) or + λΣ|w| (L1), with λ chosen on validation. Logistic regression on the 30 biopsy features; the penalty grows to the right.

SciLifeLab

L1 gives a model a pathologist can read

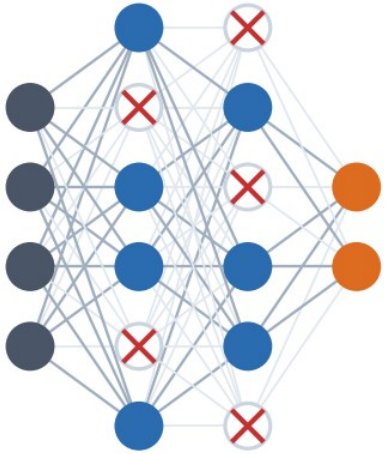


- **6 features, almost the same accuracy**
- Many features, few examples: use L1

SciLifeLab

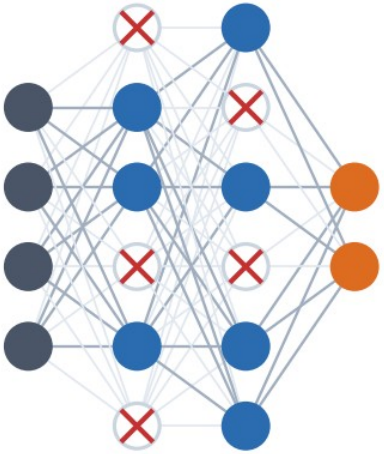
Dropout: units that cannot rely on their partners

training step 1



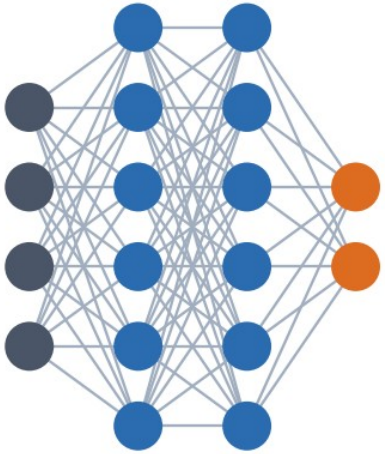
each hidden unit switched off
with probability 0.5

training step 2



each hidden unit switched off
with probability 0.5

test time: every unit on



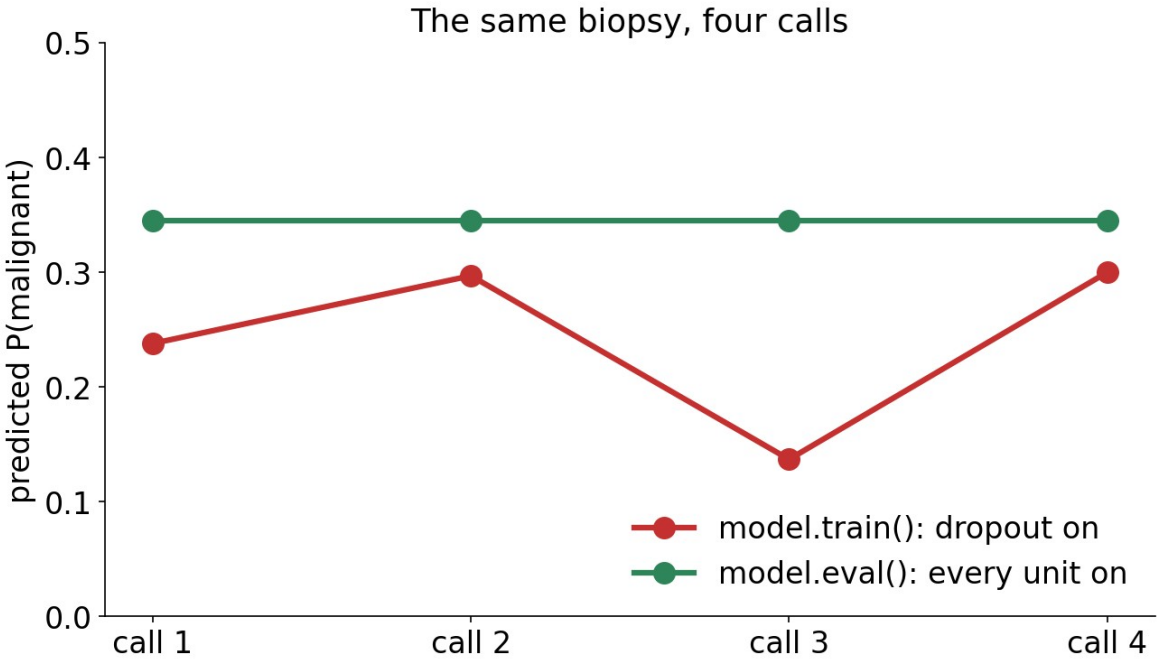
the full network acts like an
average of many thinned networks

Each hidden unit is switched off with probability p (here 0.5), with a fresh random choice for every mini-batch. At test time every unit is on.

Srivastava et al. (2014), JMLR 15:1929-1958; notebook Section 7.

SciLifeLab

Forget model.eval() and the answer changes every time



- **model.train(): dropout on**
- **model.eval(): dropout off**
- Survivors scaled by 1/(1−p)

Notebook, Section 7.

SciLifeLab

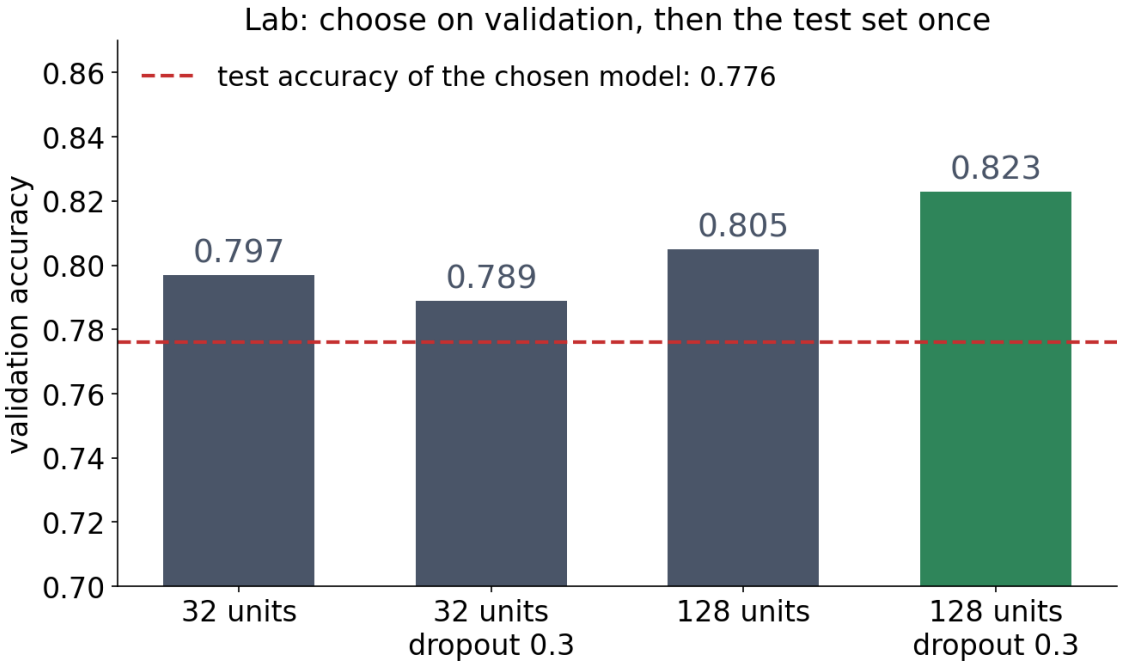
Three datasets, three answers

Breast cancer	GB1 fitness	Localisation
398 examples 513 weights	20,000 examples 9,409 weights	486 examples 192,389 weights
0.78 per weight	*2.1 per weight*	*0.003 per weight*
tie CV 0.979 vs 0.979	network R^2 0.787 vs 0.496	linear acc. 0.514 vs 0.569

Breast cancer: nearly additive, so a tie. GB1: non-additive and enough data, so the network wins. Localisation: data scarce relative to the model, so the network memorises and loses.

See course.bioinfo.se/session09.html, 'Is the improvement real?'.

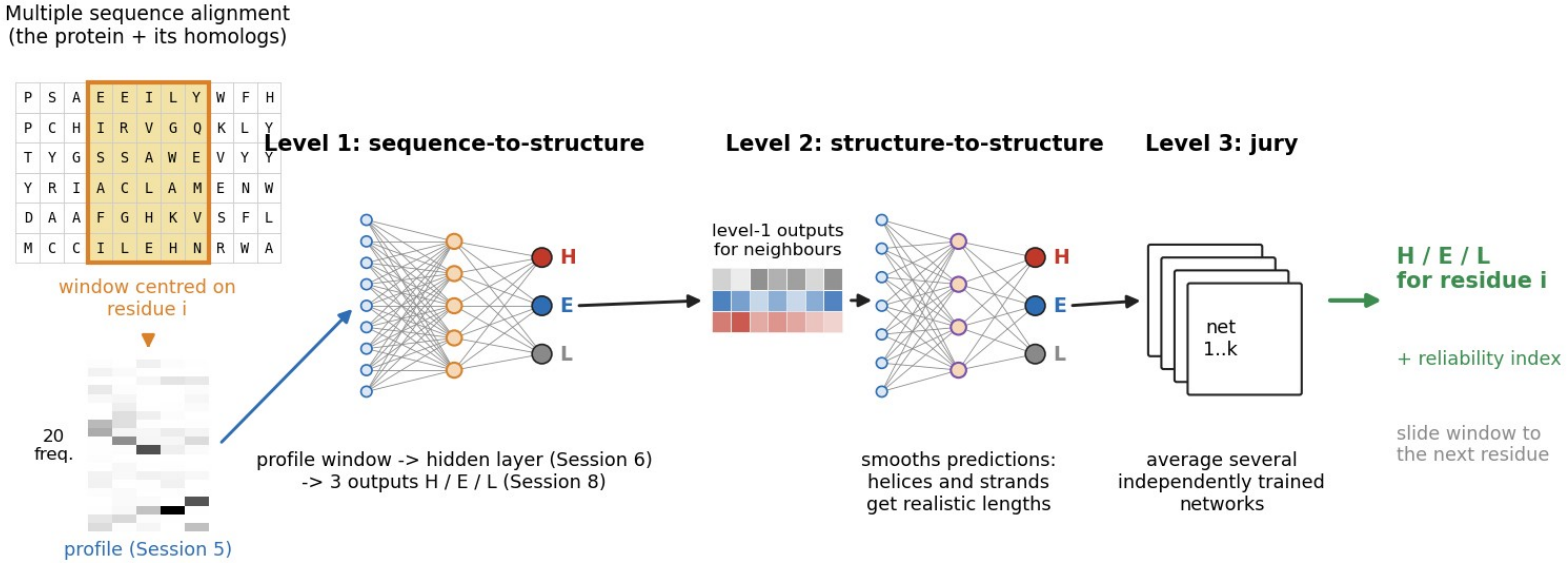
Choose on validation, test once



- Four settings tried
- Best on validation: 0.823
- **Test, used once: 0.776**

Session 9 lab, executed solution (BloodMNIST).

PHD is Sessions 5-9 in one network

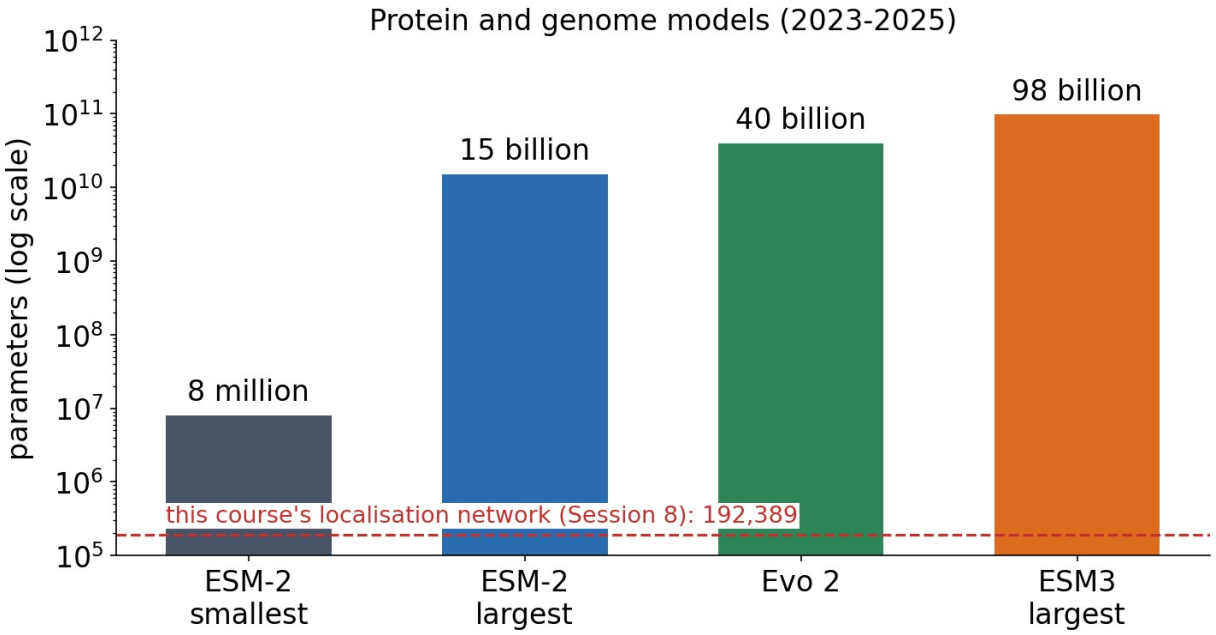


Trained by backpropagation (Session 9); tested by cross-validation on sets purged of sequences similar to the training set (Session 8's homology partitioning).

Profile input (Session 5), hidden layers (Session 7), three outputs (Session 8), trained by backpropagation (this session) and tested on sets purged of similar sequences (Session 8). 70.8% three-state accuracy.

Rost & Sander (1993), J Mol Biol 232:584-599.

Hardly anyone trains from scratch any more



- Pretrain one huge model, fine-tune it
- Bigger models, more data: lower loss
- ESM3: 10^{24} operations to train
- **Who can afford the next model, and what does that mean for open science?**

Lin et al. (2023), Science 379:1123; Hayes et al. (2025), Science 387:850; Brixi et al. (2026), Nature; Kaplan et al. (2020), arXiv:2001.08361; Hu et al. (2021), LoRA, arXiv:2106.09685.

SciLifeLab

What does the network print without model.eval()?

A trained network with dropout (p = 0.5) predicts the same biopsy five times.

```
model = nn.Sequential(
    nn.Linear(30, 64), nn.ReLU(),
    nn.Dropout(p=0.5), nn.Linear(64, 1))
train(model)                # 200 epochs

# no model.eval() here
with torch.no_grad():
    for i in range(5):
        p = torch.sigmoid(model(biopsy))
        print(f"{p.item():.2f}")
```

A The same probability five times

B Five different probabilities

C An error: dropout cannot predict

D 0.5 five times: dropout halves outputs

Guess on your own

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

What does the network print without model.eval()?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

What does the network print without model.eval()?

Fix it, run it, discuss

Open the notebook: scan the code

Fix the line marked FIX, then run the cells

Did it come out as you guessed? Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-3.ipynb

SciLifeLab

What does the network print without model.eval()?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

What does the network print without model.eval()?

Results, and who explains

Both rounds side by side.

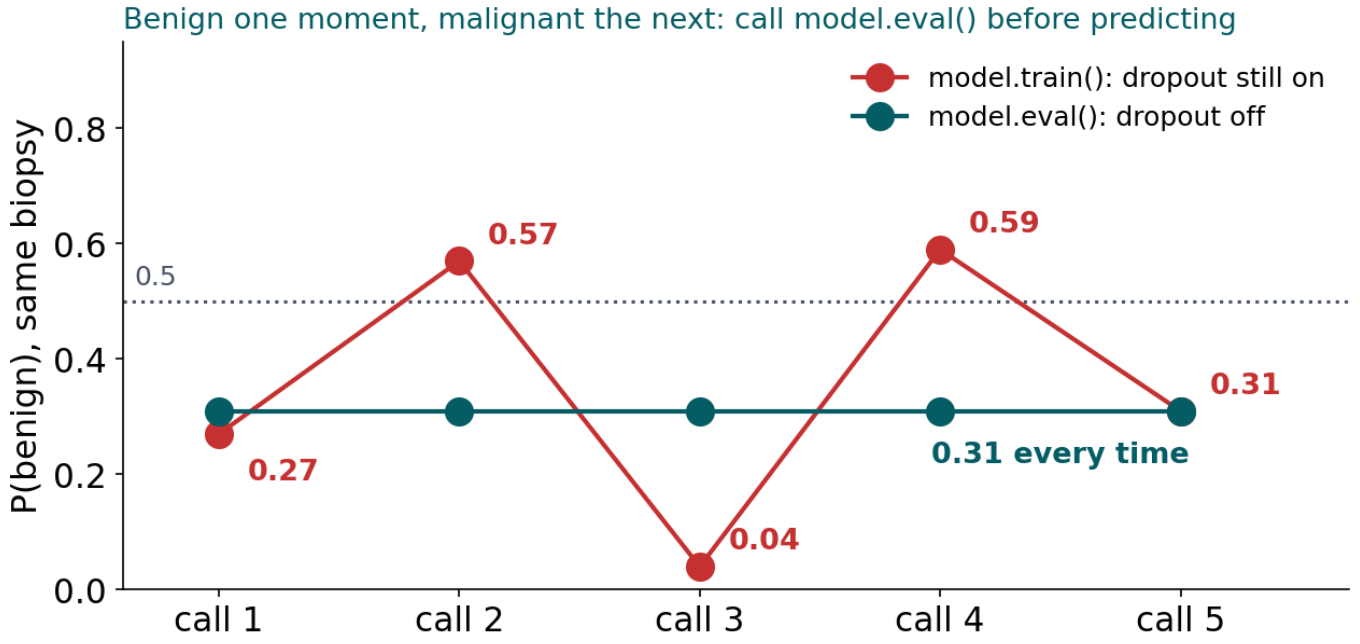
Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

To check it with code afterwards: discussion notebook 3 https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session09-discussion-3.ipynb

SciLifeLab

The answer: five different predictions, until model.eval()



In training mode, dropout drops a different random half of the hidden units at every call, so the same biopsy gets five different predictions, on both sides of 0.5. model.eval() switches dropout off: 0.31 every time. Call model.train() before training and model.eval() before evaluating.

SciLifeLab

In up to three words: what was the hardest part of today?

Training a network

loss, backprop, mini-batches

Why training fails

initialisation, vanishing gradients, epistasis

Overfitting

L1, L2, dropout, is it real?

Answer now

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

The hardest part of today

What the room found hardest

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

Session 10 next

A dense layer has a weight for every connection: 192,389 here
But the same motif means the same anywhere in a sequence: convolution